

Hybrid Search over Two-Speaker Audio Transcripts

G2 AI Hiring Hackathon: Problem Statement 1, "Effective retrieval from audio transcripts"

1. Overview

This system makes a set of two-speaker podcast and interview recordings searchable by exact words and by meaning. Each result gives the **file**, the **time range**, the **speaker**, the **exact time of the matched word**, and the turn's text with matches highlighted:

```
query: resting bothered face research

1. business_01.wav · 03:59.2–04:17.6 · SPEAKER_01 · match at 03:59.9
  There is **research** on **resting** **bothered** **face** and there are certain people who when
  people see pictures of their **face** at **rest**, they assume a mood change. ...
2. business_01.wav · 03:54.1–03:59.1 · SPEAKER_00 · match at 03:56.7
  Has anyone ever done any really compelling studies on this idea of **resting** bitch **face**
  or **resting** **bothered** **face** as you call it?
```

Everything except downloading the audio runs locally: transcription, diarization, embeddings, indexing and search.

Result: on 25 labeled queries across all seven clips, hybrid search finds the answer in the top 5 for **81%** of labeled answer locations and in the top 10 for **92%**, meeting every success criterion (section 7). Exact names and phrases are found at rank 1–3; paraphrased questions are the weak spot.

2. Architecture

audio (.wav, 16 kHz mono)

1. Transcription

2. Diarization

3. Chunking

4. Embedding

5. Indexing

WhisperX (faster-whisper "small", English) + wav2vec2 forced alignment
→ word timestamps

pyannote speaker-diarization-community-1 (via WhisperX), fixed at 2 speakers
→ speaker assigned per word; segments split where the speaker changes

one chunk = one speaker turn (≤ 80 words), with word timings

BAAI/bge-small-en-v1.5 (384-d, local): turn + neighbouring turns

PostgreSQL 16 + pgvector: HNSW (cosine) + GIN full-text index

query → keyword search (Postgres full-text, any query word) →

↳ semantic search (pgvector cosine, BGE instruction) → weighted RRF

→ file · time · speaker · highlighted turn

Component	Location
Full ingestion pipeline	src/pipeline.py
Transcription + diarization	src/diarization/diarize.py
Speaker-turn chunking	src/chunking/chunk.py
Embeddings	src/embeddings/embed.py
Database connection and writes	src/database/
Schema (applied automatically by Docker)	db/schema.sql
Hybrid search (CLI)	src/retrieval/pg_hybrid_search.py
Web UI + REST API for evaluators	src/api/app.py

Component	Location
Re-index without re-transcribing	<code>src/index_chunks.py</code>
Golden dataset builder	<code>scripts/build_dataset.py</code> , <code>data/dataset.json</code>
Labeled queries + validator	<code>data/queries.json</code> , <code>src/evaluation/validate_queries.py</code>
Evaluation, run comparison, tests	<code>src/evaluation/evaluate.py</code> , <code>src/evaluation/report.py</code> , <code>tests/</code>
Detailed evaluation and experiment log	<code>docs/evaluation.md</code>

3. Architecture decisions and technology choices

Each component below lists what was chosen, how it is configured, why, and what else was considered. "Considered" means compared on documented trade-offs, not benchmarked here, unless a measurement is given.

3.1 Summary

Layer	Choice	Key configuration
Transcription	WhisperX 3.8 (faster-whisper backend)	Model <code>small</code> , int8 on CPU (float16 on GPU), batch 16, language pinned to English
Word alignment	wav2vec2 <code>WAV2VEC2_ASR_BASE_960H</code> (WhisperX default for English)	Word-level timestamps; character alignments off
Voice activity detection	pyannote VAD (WhisperX default)	Speech merged into ≤ 30 s windows for batching
Diarization	pyannote <code>speaker-diarization-community-1</code> (pyannote.audio 4.0)	Exactly 2 speakers; word-level speaker assignment
Chunking	Speaker turns (custom)	≤ 80 words per turn; ± 1 neighbouring turn as embedding context; turns under 8 words embedded alone
Embeddings	BAAI/bge-small-en-v1.5 via sentence-transformers	384-d, L2-normalised, batch 32, CPU; BGE query instruction on queries only
Database	PostgreSQL 16 + pgvector 0.8, in Docker (pgvector/pgvector:pg16)	HNSW (cosine) + GIN full-text; schema applied on first start
Keyword search	Postgres full-text, <code>english</code> configuration	Any query word matches (OR); quoted text = exact phrase; <code>ts_rank</code> without length normalisation
Semantic search	pgvector cosine distance	Turns under 4 words excluded
Fusion	Weighted Reciprocal Rank Fusion	$k = 60$, keyword weight 0.5, top 20 candidates per retriever
Dataset tooling	yt-dlp + ffmpeg	Clips cut from a manifest, 16 kHz mono WAV

3.2 Why each choice

Transcription: WhisperX

- **Word-level timestamps.** Plain Whisper's segment times drift by seconds; WhisperX adds wav2vec2 forced alignment, giving every word its own start and end. That is what makes `match at mm:ss` possible.
- **Fast and local on CPU.** faster-whisper with int8, and voice activity detection first, so Whisper only sees speech (fewer hallucinations over silence). No per-hour cost; audio never leaves the machine.

- **Trade-off:** `small` misspells some names, which hurts exact keyword search; a larger model is a one-line change.

Diarization: `pyannote speaker-diarization-community-1`

- Local, open, and WhisperX's default. The speaker count is fixed at two, as the brief guarantees, which stops one voice being split into several.
- Speakers are assigned **per word**, and segments are split wherever the speaker changes; a one-word flip is treated as noise and smoothed away.

Chunking: one speaker turn per chunk

- A chunk is consecutive segments by one speaker, split at sentence boundaries past 80 words (240 turns, median 16 s). Each result therefore has one speaker and a short time range; the first prototype's ~100 s windows mixed both speakers (issue 2).
- **Two texts per chunk.** `text` (the turn) is displayed and keyword-indexed; `embed_text` (the turn plus one turn either side) is embedded, so an answer's vector includes the question it answers. The longest `embed_text` is 299 tokens, within the model's 512.
- **Short turns.** Turns under 8 words embed only their own text, so filler doesn't borrow its neighbours' meaning; turns under 4 words ("Look.", "What?") are left out of semantic search entirely (issue 11).
- **Rejected:** merging short turns into neighbours (misattributes speakers) and removing neighbouring results at search time (hid the correct turn, issue 8).

Embeddings: `BAAI/bge-small-en-v1.5`

- Runs locally as the brief requires, strong on retrieval benchmarks for its size, and small: 384 dimensions, all 240 turns indexed in about 12 s on CPU.
- **Queries get BGE's documented instruction** ("Represent this sentence for searching relevant passages: "); turns do not. This was the largest single improvement: semantic-only recall@10 0.848 → 0.917.

Database: PostgreSQL 16 + pgvector in Docker

- **One system** for vectors, full-text search, metadata and transactions, so both searches filter the same way and re-indexing a file is one transaction. The schema is applied automatically on first start.
- **Indexes:** HNSW (cosine) on embeddings; a generated `tsvector` column with a GIN index for keyword search; per-word timings in `words` (JSONB) for the match time.
- **Gap:** `ts_rank` is not BM25 (no inverse document frequency); ParadeDB's `pg_search` would add BM25 inside Postgres.

Search and fusion

- **Keyword:** any query word can match (OR), ranked by `ts_rank`, so natural-language questions still match (issue 3); "quoted text" is an exact phrase. No length normalisation, which pushed one-word turns to the top (issue 12).
- **Fusion: weighted Reciprocal Rank Fusion**, $1/(60 + \text{semantic rank}) + 0.5/(60 + \text{keyword rank})$ over the top 20 of each. RRF uses only ranks, so cosine and `ts_rank` scores never need to be made comparable. Keyword ranks count half because OR matching also returns turns that share only common words (issues 10 and 14).
- **Results** show the whole turn with matched words highlighted, and the time of the first matching word.

3.3 Alternatives considered

Layer	Alternatives	Why not chosen
Transcription	OpenAI Whisper; hosted APIs (AssemblyAI, Deepgram); NeMo / Parakeet	No reliable word timings; cost and audio leaving the machine; heavier GPU setup
Embeddings	all-MiniLM-L6-v2; bge-base / e5-large	256-token limit too short for turn + context; 3–10× slower (a candidate for improving paraphrase recall)
Database	Qdrant / Weaviate / Milvus; Elasticsearch; SQLite + sqlite-vec	A second system to keep in sync; heavy for this scale; weaker full-text ranking
Fusion	Weighted score blending; cross-encoder reranker	Needs score normalisation and tuning; a reranker is the next step, not yet built

4. Golden dataset

Seven clips, 8–10 minutes each (64 minutes total), a different pair of speakers in each file, cut from long-form podcast interviews and grouped into topic themes.

File	Show	Speakers	Topic	Length
tech_ai_01.wav	Dwarkesh Podcast	Dwarkesh Patel, Andrej Karpathy	AI research	10.3 min*
tech_product_01.wav	Lenny's Podcast	Lenny Rachitsky, Ramesh Johari	Product / marketplaces	8.5 min
business_01.wav	Diary of a CEO	Steven Bartlett, Vanessa Van Edwards	Charisma / communication	8.6 min
business_02.wav	Acquired	Ben Gilbert / David Rosenthal (hosts), Jensen Huang	Luck & skill / job displacement	9.0 min
health_01.wav	Huberman Lab	Andrew Huberman, Dr. Chris Thompson	Gut microbiome / fasting	8.5 min
health_02.wav	The Drive (Peter Attia)	Peter Attia, Ralph DeFronzo	Insulin resistance / type 2 diabetes	9.2 min
culture_01.wav	Fresh Air (NPR), 1997	Terry Gross, Diane Keaton	Film / directing	9.0 min

* Slightly over the 10-minute target; the last minute is an ad read with no labeled answers.

- **Build.** `scripts/build_dataset.py` downloads each episode with `yt-dlp`, cuts the section defined in `data/dataset.json` with `ffmpeg`, and converts it to 16 kHz mono WAV.
- **Checks.** It verifies each clip is 8–10 minutes and warns if a speaker appears in more than one clip.
- **Reproducibility.** The manifest records the source URL, clip start and duration for every file. The transcripts are included in `output/transcription/`, so the index can be rebuilt without a GPU or Hugging Face token.

Labeled query set (`data/queries.json`): **25 queries, 3–4 per file.**

Type	Count	What it tests
keyword	6	Rare words and names said once (e.g. "Barry Marshall", "oDesk")
phrase	3	Quoted exact wording (e.g. "euglycemic clamp")
semantic	10	Paraphrases that avoid the speaker's distinctive words (e.g. "why shots given under the skin don't mimic the body's own delivery route")
speaker	3	"What did the host say about X", where both speakers discuss X; only the host's turn counts
negative	3	Topics no file covers; one shares a word with a file ("sourdough bread")

- **Labels.** Each query is labeled with the file(s) and time range(s) where the answer is said, plus the speaker; 9 queries have 2–3 separate locations.
- **Labelling rule.** One continuous answer by one speaker is a single time range, even if it spans several chunks.
- **Validation.** `python -m src.evaluation.validate_queries` checks ids, types, time ranges against each file's duration and speakers, and warns when a semantic query reuses words from its answer (issue 13).
- **Not covered:** no query has answers in more than one file, so cross-file retrieval is untested.

5. Evaluation method

- **Match rule.** A result is correct if it comes from the labeled file and **overlaps the labeled time range** by at least 0.5 s (or half the result, for very short turns). Labels are time ranges, not chunk IDs, so they stay valid when chunking changes.
- **Retrievers compared.** Every query is run through **keyword-only, semantic-only and hybrid** search, so the benefit of fusion is measured, not assumed.
- **Runs are saved and compared.** Each run is saved with the configuration and git commit that produced it; `report.py` prints the tables below and the change against the baseline.

Metric	Definition
recall@k (k = 1, 3, 5, 10)	Fraction of labeled answer locations found in the top k, averaged over queries
MRR	Mean of 1 / rank of the first correct result (within the top 10)
recall by query type	Shows where keyword vs semantic retrieval matters
Speaker accuracy	Share of queries whose first correct hybrid result (top 10) has the labeled speaker
Top semantic score, positive vs negative queries	Evidence for or against a "no good match" threshold

```
python -m src.evaluation.evaluate --out output/eval/<run>.json
python -m src.evaluation.report output/eval/<run>.json --compare output/eval/baseline.json
```

6. Success criteria

Criterion	Target	Why
Hybrid recall@5	≥ 0.80	A user scanning the first screen of results finds the moment
Hybrid recall@10	≥ 0.90	The answer is almost always retrievable
Speaker accuracy	≥ 0.80	Results must attribute the words to the right person
Hybrid ≥ keyword-only and ≥ semantic-only (recall@5)	Holds	Fusion must justify itself over either method alone
Every result shows file, time range and speaker	Always	Required by the brief

The first four are computed by `evaluate.py` and reported by `report.py`. The fifth was checked over the top 10 results of all 25 queries (250 of 250 have file, time range and speaker). The matching and metric code has unit tests in `tests/test_metrics.py`; the criteria are not yet enforced as a pytest gate against the live index.

7. Results

All numbers are over the 22 labeled queries (the 3 negatives have nothing to recall). One query is worth about 0.045 of recall.

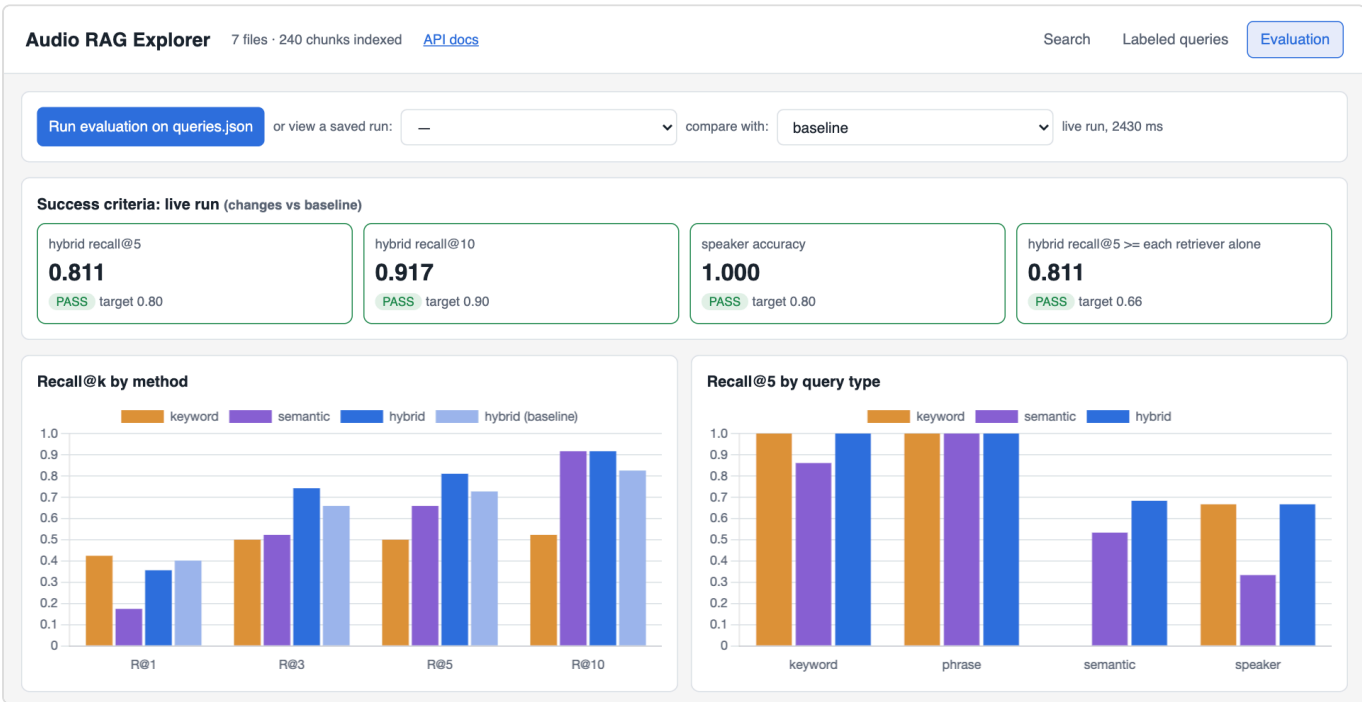
7.1 Final results

Criterion	Target	Result	Status
hybrid_recall@5	≥ 0.80	0.811	PASS
hybrid_recall@10	≥ 0.90	0.917	PASS
speaker_accuracy	≥ 0.80	1.000	PASS
hybrid recall@5 ≥ each retriever alone	holds	0.811 vs 0.500 / 0.659	PASS
every result shows file, time range and speaker	always	250 / 250	PASS

Change from the baseline in brackets:

Method	R@1	R@3	R@5	R@10	MRR
Keyword (text)	0.424	0.500	0.500	0.523	0.505
Semantic	0.174 (-0.091)	0.523 (+0.061)	0.659 (+0.030)	0.917 (+0.068)	0.508 (-0.016)
Hybrid (RRF)	0.356 (-0.045)	0.742 (+0.083)	0.811 (+0.083)	0.917 (+0.091)	0.619 (+0.008)

The same results in the evaluation UI (section 11.2), compared with the baseline:



Evaluation tab: success criteria and recall@k by method and query type, with the baseline overlaid

7.2 Baseline (before the retrieval improvements)

Unweighted RRF, no query instruction. Saved as `output/eval/baseline.json`.

Method	R@1	R@3	R@5	R@10	MRR
Keyword (text)	0.424	0.500	0.500	0.523	0.505

Method	R@1	R@3	R@5	R@10	MRR
Semantic	0.265	0.462	0.629	0.848	0.524
Hybrid (RRF)	0.402	0.659	0.727	0.826	0.611

Type (n)	Key R@1	Key R@3	Key R@5	Key R@10	Sem R@1	Sem R@3	Sem R@5	Sem R@10	Hyb R@1	Hyb R@3	Hyb R@5	Hyb R@10
keyword (6)	0.81	1.00	1.00	1.00	0.22	0.56	0.69	1.00	0.81	1.00	1.00	1.00
phrase (3)	0.83	1.00	1.00	1.00	0.50	0.67	1.00	1.00	0.83	1.00	1.00	1.00
semantic (10)	0.00	0.00	0.00	0.05	0.20	0.38	0.47	0.67	0.05	0.35	0.50	0.62
speaker (3)	0.67	0.67	0.67	0.67	0.33	0.33	0.67	1.00	0.33	0.67	0.67	1.00
all (22)	0.42	0.50	0.50	0.52	0.27	0.46	0.63	0.85	0.40	0.66	0.73	0.83

Criterion	Target	Result	Status
hybrid_recall@5	≥ 0.80	0.727	FAIL
hybrid_recall@10	≥ 0.90	0.826	FAIL
speaker_accuracy	≥ 0.80	1.000	PASS
hybrid recall@5 ≥ each retriever alone	holds	0.727 vs 0.500 / 0.629	PASS

What the baseline showed:

- **Exact words worked.** Every keyword and phrase query was found in the hybrid top 3, and names at rank 1.
- **Paraphrases were the weak spot:** semantic-type hybrid recall@5 was 0.50.
- **Fusion both helped and hurt.** Hybrid had the best R@3, R@5 and MRR, but on four queries (q02, q18, q21, q22) keyword matches on common words pushed the answer below where semantic search alone had it, so semantic beat hybrid at R@10 (0.848 vs 0.826).

7.3 Improving retrieval

Variants were scored on the same queries against the baseline. The kept one is in bold.

Change	Hybrid R@5	Hybrid R@10	MRR	Notes
Baseline	0.727	0.826	0.611	
Keyword weight 0.5 or 0.7 in RRF	0.758	0.826	0.616	fixes the near-misses only partly
Keyword match must cover ≥ 2 query words	0.758	0.826	0.678	best MRR without the query instruction
BGE query instruction, plain RRF	0.788	0.811	0.600	semantic-only R@10 0.848 → 0.917
BGE query instruction + keyword weight 0.5	0.811	0.917	0.619	only variant passing both recall targets
BGE query instruction + keyword ≥ 2 words	0.742	0.871	0.669	
BGE query instruction + keyword ≥ half the query words	0.697	0.917	0.716	best MRR, worst R@5

Kept: the BGE query instruction (the model's documented usage, not a tuned value) plus keyword weight 0.5 in fusion. Neither needs re-indexing.

Recall by query type after the change:

Type (n)	Key R@1	Key R@3	Key R@5	Key R@10	Sem R@1	Sem R@3	Sem R@5	Sem R@10	Hyb R@1	Hyb R@3	Hyb R@5	Hyb R@10
keyword (6)	0.81	1.00	1.00	1.00	0.22	0.53	0.86	1.00	0.81	1.00	1.00	1.00

Type (n)	Key R@1	Key R@3	Key R@5	Key R@10	Sem R@1	Sem R@3	Sem R@5	Sem R@10	Hyb R@1	Hyb R@3	Hyb R@5	Hyb R@10
phrase (3)	0.83	1.00	1.00	1.00	0.17	0.67	1.00	1.00	0.83	1.00	1.00	1.00
semantic (10)	0.00	0.00	0.00	0.05	0.20	0.53	0.53	0.82	0.05	0.53	0.68	0.82
speaker (3)	0.67	0.67	0.67	0.67	0.00	0.33	0.33	1.00	0.00	0.67	0.67	1.00
all (22)	0.42	0.50	0.50	0.52	0.17	0.52	0.66	0.92	0.36	0.74	0.81	0.92

Reading the change:

- **Paraphrase queries gained most:** semantic-type hybrid recall@5 0.50 → 0.68. q15 went from missed to rank 3, q22 from 10 to 4, q18 from 6 to 3, q11 from 4 to 2.
- **Rank 1 got slightly worse:** hybrid R@1 0.402 → 0.356, and q03, q09 and q12 each dropped one place. With the instruction, semantic search is better at getting the answer into the top few and worse at putting it first.
- **Names and phrases were unaffected:** still found by rank 3.
- **Still missed in the top 5:** q02 (rank 10), q17 (not in the top 10) and q21, the host-intent query (rank 10).
- **Caveat:** the kept variant was chosen from about a dozen scored on these same 22 queries, so the gain is likely somewhat optimistic. It should be confirmed on queries written after this change.

7.4 Negative queries and a relevance threshold

Mean top semantic score is **0.56** for the negative queries and **0.67** for real ones, but the ranges overlap: negatives score 0.54–0.56, while the name query "Barry Marshall" scores 0.49 (names carry little meaning for an embedding model). A threshold on the semantic score alone would reject valid name searches, so it would need to be combined with "no keyword match" before it could hide off-topic results.

8. How the solution evolved: issues found and fixes

Chronological record of the problems found while building and testing the system, and what was changed. Issues 1–9 come from the first prototype; the system was then rebuilt step by step in this repository, and issues 10–14 were found during that rebuild and its evaluation.

Starting point. The first working version had:

- chunks of about 300–400 words (about 100 s) containing both speakers;
- a `speaker` field like "SPEAKER_00, SPEAKER_01";
- AND-only keyword search;
- no evaluation.

It returned the right file but not a usable timestamp or speaker.

#	Issue found	How it showed up	Fix
1	ASR, alignment and diarization models reloaded for every file	Code review of the multi-file pipeline	Model loaders cached (<code>lru_cache</code>), so each model loads once per run
2	Results couldn't identify the speaker or moment	Review against the brief: ~100 s chunks, both speakers per chunk, word timings discarded	Redesigned around speaker turns: word timings kept, segments split on word-level speaker changes, one chunk per turn, context-aware embeddings. The same 5-minute file went from 3 chunks of ~100 s to 25 turns (median 7.6 s)
3	Multi-word keyword queries often returned nothing	<code>plainto_tsquery</code> requires every term	Terms OR-ed; quoted input uses exact phrase matching

#	Issue found	How it showed up	Fix
4	Re-running a file left stale rows; one DB connection per chunk	Code review	Delete-then-insert per file in one transaction; idempotent schema; clean re-index
5	Filler turns ("Oh, I see. They make the") ranked for unrelated queries	Manual test queries on a real file	Neighbour context made them borrow their neighbours' meaning. Turns under 8 words now embed only their own text. Merging them into neighbours was tested and rejected: it would mislabel real short turns such as "How old are you?"
6	Changing chunking meant re-transcribing everything	Iteration speed	<code>src/index_chunks.py</code> re-chunks, re-embeds and re-indexes from saved transcripts
7	Evaluation counted the wrong turn as correct	Speaker accuracy failed; the "correct" hits were the other speaker's adjacent turn	A ± 2 s tolerance accepted neighbouring turns, which sit back to back. Replaced with a real-overlap requirement. Scores dropped to their true values
8	De-duplicating neighbouring results hid the correct turn	After fix 7, hybrid recall@5 was 0.44 vs semantic 0.94. Inspection showed the correct turn (semantic rank 1) removed in favour of its neighbour	De-duplication disabled; hybrid recall@5 rose to 0.81 (0.88 after 9)
9	Answers spanning two chunks were labelled as two spans	Recall capped at 0.5 for those queries	Labelling rule: one continuous answer by one speaker = one time range
10	Hybrid ranked below semantic on some paraphrase queries	q07 (prototype set): semantic rank 1, missing from hybrid top 10	OR-ed keyword matching lets common words match many turns, and equal-weight RRF trusted those matches. Deferred until the full query set existed; fixed in issue 14
11	Semantic top results filled with one-word turns	Test query "marketplace": top 3 were "Look.", "What?", "Oh, what"	Near-empty embeddings score moderately against any query. Turns under 4 words are excluded from semantic search (8 of 240); they stay keyword-searchable
12	Keyword search ranked one-word turns first	After OR-ing, "how to look genuinely happy in a profile photo" returned "Look." at keyword rank 1	<code>ts_rank</code> length normalisation favoured tiny turns. Turned off: across four test queries no short turn remained in the keyword top 5 and the hybrid #1 was unchanged
13	Evaluation queries reused the speakers' wording	Validator warning: semantic queries sharing content words with their labeled answer	Four queries rewoded; the validator now warns on any overlap, so "semantic" queries really test embeddings
14	Recall targets missed on the full query set	Baseline: hybrid recall@5 0.727, recall@10 0.826, below semantic-only at R@10	BGE query instruction on queries + keyword weight 0.5 in RRF: recall@5 0.811 , recall@10 0.917 (section 7.3)

9. Limitations

Where retrieval is expected to fail, with evidence in `docs/evaluation.md`:

- **Paraphrases with no words in common with the speaker.** The small embedding model still misses some meaning-only matches, especially technical ones (q17 is not in any retriever's top 10).
- **Host / guest intent.** Speaker labels are anonymous (`SPEAKER_00` / `SPEAKER_01`) and search has no notion of roles, so "what did the host say about X" is answered from content only and can return the guest's turn first (q21 at rank 10). Nothing links speaker labels to names either, so "what did Karpathy say" cannot filter to Karpathy's turns.

- **Names the transcription got wrong.** Whisper `small` misspells names, and full-text search needs the exact word: DeFronzo, James Caan and Gerald Reaven return nothing because they were transcribed as "DeFranco", "Jimmy Khan" and "Jerry Reven". The trigram index could catch these but is not yet used.
- **Diarization errors.**
 - pyannote sometimes splits a sentence between speakers; the sentence is then cut across two turns ("... And you" / "think, Ralph, that...").
 - Overlapping speech gets a single speaker, and the two-speaker count is fixed, not detected.
- **Very short turns** (under 4 words) can only be found by keyword.
- **Queries made only of common words** ("we are the") match nothing by keyword, because the `english` configuration drops stop words.
- **No relevance threshold.** An off-topic question ("who won the 2022 football world cup") still returns five results; see section 7.4 for why a simple score cutoff is not enough.
- **Answers spread across files** are untested, and neighbouring turns cluster in the top results, so a second answer location far away in time can fall below the top k.
- **Small, agent-drafted evaluation set.** 22 labeled queries, drafted by the coding agent from the transcripts and reviewed by the author; the final configuration was selected on the same set. English only.
- **CPU processing.** Transcription and diarization take about 12 minutes per 9-minute clip on a laptop CPU. Fine for this dataset; production would need GPUs or a hosted ASR service.

10. Taking it to production

The brief asks which metrics matter if this system went to production, and how to evaluate it there.

What to measure:

What	Metric	Why it matters
Search quality	recall@5 and MRR on the labeled queries, re-run after every change	Catches a change that makes search worse before users see it
Transcription quality	Word error rate on a sample checked by a person	A misheard word can't be found by keyword search
Speaker labels	How often the wrong speaker is assigned (diarization error rate)	Wrong labels give wrong answers to "what did the guest say"
Speed	Search time per query, typical and worst case	Users expect results in well under a second
Real usage	How often users play a result, or rephrase their query instead	Shows success on real questions, not just the labeled ones

How to evaluate it:

- **Before each release:** run the labeled queries automatically and block the change if recall drops. The comparison already exists (`report.py --compare`); it only needs wiring into CI.
- **After release:** log which results people play; periodically have a person judge a sample of real queries and add them to the labeled set, so it keeps up with what users actually ask.
- **Separately for each stage:** measure transcription and speaker labels on their own, so a drop in search quality can be traced to its cause.

At larger scale: transcription moves to GPU workers, and searches filter by file or speaker before the vector search.

11. Running it

11.1 Setup and scripts

Requires Python 3.12 or 3.13, Docker, and ffmpeg (only to transcribe). The Hugging Face token is needed only to transcribe/diarize: accept the terms of `pyannote/speaker-diarization-community-1` and set `HF_TOKEN` in `.env`. Everything else works without it because the transcripts are included.

```
python3 -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt
cp .env.example .env
docker compose up -d                                # Postgres + pgvector; schema from db/schema.sql

python -m src.index_chunks                          # index the included transcripts (~12 s, no GPU)
python -m src.pipeline                              # or: transcribe + index from the audio (reuses
transcripts)
python scripts/build_dataset.py                      # (optional) rebuild data/audio/ from data/dataset.json

python -m src.retrieval.pg_hybrid_search "insulin resistance in lean people"
python -m src.evaluation.evaluate                    # recall@k report -> output/eval/results.json
python -m src.evaluation.report output/eval/results.json --compare output/eval/baseline.json
pytest                                               # unit tests

uvicorn src.api.app:app                             # web UI + API (section 11.2)
```

11.2 Web UI and API for evaluators

`uvicorn src.api.app:app` serves a web UI at `http://localhost:8000` and a REST API documented at `/docs`, where every endpoint can be tried in the browser. It uses the same search and evaluation code as the scripts, so a live run in the UI reproduces the numbers in section 7.

Tab	Purpose
Search	Hybrid results with file · time · speaker and matched words highlighted; ▶ plays the audio from the matched word. The keyword-only and semantic-only lists are shown underneath, so the effect of fusion is visible.
Labeled queries	Pick one of the 25 labeled queries to see whether each method found its known answer and at what rank (correct result in green), with recall@k. A form checks your own query, scored if you give the file and time range of the answer.
Evaluation	Runs all labeled queries (~2 s): success criteria, recall@k by method and query type, a per-query table, and a comparison with any saved run such as the baseline.

Audio RAG Explorer

7 files · 240 chunks indexed

API docs

Search

Labeled queries

Evaluation

insulin resistance in lean people

5

All files

Search

Hybrid = semantic (pgvector) + keyword (Postgres full-text) merged with weighted RRF. Put text in "double quotes" for an exact phrase. ▶ plays from the matched word.

Hybrid results (2025 ms)

#1 health_02.wav · 08:50.0–08:57.0 · SPEAKER_01 · match at 08:50.5 ▶ 08:50.5

score 0.0241

But these people were all lean and they're insulin resistant. So there's a genetic cause of the insulin resistance. And you

#2 health_02.wav · 08:57.1–09:09.8 · SPEAKER_00 · match at 09:02.2 ▶ 09:02.2

score 0.0238

think, Ralph, that the, the greater genetic effect is on the insulin resistance side or on the beta cell fatigue side. Both. Okay. So let's tackle each. And since you started with insulin resistance, let's go there.

#3 health_02.wav · 05:20.5–05:52.9 · SPEAKER_00 · match at 05:30.1 ▶ 05:30.1

score 0.0230

And you've obviously had to bring glucose. You had to infuse glucose to maintain euglycemia. Correct. Did you say that in four days? 48 to 72 hours. And these people are as insulin resistant as type 2 diabetics. Okay. Again, very, very counterintuitive. Because if our model is that insulin resistance which is the hallmark factor contributing to type two diabetes in the combination of beta cell fatigue is driven by lipotoxicity, which we're going to come to.

#4 health_02.wav · 03:40.5–04:08.6 · SPEAKER_01 · match at 03:42.4 ▶ 03:42.4

score 0.0227

And we raised their fasting insulin from eight, which is what a normal person would be, to 20, which is really quite low. And within 48 to 72 hours, they were as insulin resistant as a type 2 diabetic patient. So hyperinsulinemia induces insulin resistance. This is unacceptable. Wait a second, why is that the case? So what insulin does is it down regulates the insulin signaling transduction system.

#5 health_02.wav · 05:53.0–06:08.9 · SPEAKER_00 · match at 05:55.3 ▶ 05:55.3

score 0.0223

That's an important one. These people didn't have any of that. These people didn't have any of the intramiocellular lipid that we talked about with your colleague, Jerry Reven, as a predisposing factor. It's the direct effect of insulin

Search tab: hybrid results with highlighted matches, speaker and timestamp, and audio playback

Audio RAG Explorer

7 files · 240 chunks indexed

API docs

Search

Labeled queries

Evaluation

Check retrieval against a known answer. Each labeled query has a moment where the answer is actually said. Pick one to see whether keyword, semantic and hybrid search found that moment, and at what rank (the correct result is highlighted green), plus its recall@k. This is the drill-down behind the Evaluation tab. To just ask a question, use Search.

Labeled queries

All types

id	type	query
q01	keyword	Barbara Wilde
q02	semantic	believing others are upset with you when their face is just blank
q03	speaker	what did the host say about people who look grumpy even when they're happy
q04	keyword	Getty and Shutterstock
q05	phrase	"infinite ambition"
q06	semantic	will smarter computers leave lots of people unemployed
q07	keyword	Jillian Armstrong
q08	semantic	feeling isolated among male co-stars in a mob film
q09	semantic	being timid about bossing performers around behind the camera
q10	keyword	Barry Marshall
q11	semantic	what do gut microbes make that nourishes the lining of the large intestine
q12	speaker	what did the host say about stress and stomach ulcers
q13	phrase	"euglycemic clamp"
q14	semantic	a short experiment that made fit young adults metabolically sick within days
q15	semantic	why shots given under the skin don't mimic the body's own delivery route
q16	keyword	NanoChat
q17	semantic	why automated programming helps struggled with his unusual project
q18	semantic	has software been gradually taking over coders' grunt work for decades

q15 · semantic (707 ms)

why shots given under the skin don't mimic the body's own delivery route

Injected insulin goes to the periphery; natural insulin passes the liver via the portal vein first. Avoids 'portal', 'liver', 'periphery'.

Labeled answer:

- health_02.wav · 06:49.7–07:47.6 · SPEAKER_01 ▶ play answer

method	R@1	R@3	R@5	R@10	RR	first hit
keyword	0.00	0.00	0.00	0.00	0.00	not in top 10
semantic	0.00	1.00	1.00	1.00	0.33	rank 3
hybrid	0.00	1.00	1.00	1.00	0.33	rank 3

hybrid top 10

#1 business_01.wav · 03:54.1–03:59.1 · SPEAKER_00 ▶ 03:54.1

score 0.0164

Has anyone ever done any really compelling studies on this idea of resting bitch face or resting bothered face as you call it?

#2 health_02.wav · 07:48.4–08:14.3 · SPEAKER_01 ▶ 07:48.4

score 0.0161

And in a certain way, if you're a diabetic and you are insulin resistant or an obese person and you are insulin resistant and you're hyper secreting insulin, it's kind of working against you because it's a reverberating system that's making the insulin resistance aggravated. So, one of the big things that we've forgotten is that insulin, I told you, there are two problems in diabetes. One is you don't make enough insulin.

#3 health_02.wav · 06:56.2–07:17.5 · SPEAKER_01 ▶ 06:56.2

labeled answer

score 0.0159

And you or I, when you ingest the meal, where does the insulin go? It goes into the portal vein. So the liver is seeing a high level of insulin. That's good. It says stop making glucose. But now it removes half of the insulin. how much insulin gets into the periphery, half of what you've secreted. Why?

#4 health_01.wav · 06:13.4–06:23.0 · SPEAKER_00 ▶ 06:13.4

score 0.0156

And that's the problem with these, you know, the way the media covers these findings, like it's not all that you ingested the wrong soil. Stress can give you ulcers, right? Or am I missing something?

#5 health_01.wav · 06:23.5–06:41.5 · SPEAKER_01 ▶ 06:23.5

score 0.0154

stress can play a role, it is certainly complicated. So Barry Marshall was phenomenal in Australia and he found H. Pylori could cause gastric ulcers, right? And he had to consume, no one believed him. He had to consume him himself, consume it, and then he had gastric ulcers. I love it when scientists take self-experimentation. That's crazy, right? Yeah.

Labeled queries tab: q15's known answer is found at rank 3 by semantic and hybrid search, missed by keyword search

Endpoint	Returns
GET /search?q=&k=&file=	Hybrid results plus the keyword-only and semantic-only lists
GET /queries	The labeled evaluation queries
GET /evaluate/query/{id}	One labeled query: each method's top 10, which results hit the answer, recall@k
POST /evaluate/query	The same for a custom query, with or without an answer span
POST /evaluate	Runs all labeled queries: metrics, success criteria, per-query detail
GET /results, GET /results/{run}	Saved evaluation runs (baseline, exp1,...)
GET /audio/{file}	The audio clip, with seeking (used to play from a timestamp)
GET /health, GET /meta	Database status and what is indexed; constants the UI uses

12. How I used a coding agent

I built this with **Claude Code** (Anthropic) as my coding agent. I use it the way I would use a fast, capable teammate: I own the design, the order of work and every commit; the agent writes most of the code, runs it, and reports back with evidence.

Plan first, then small, specified steps.

- **A written build plan.** I broke the work into 13 numbered steps: scaffold → diarization → chunking → embeddings → schema → repository → indexing → hybrid search (in stages) → labeled queries → evaluation → tests → pipeline.
- **A short spec per step.** Each step listed the files and functions to create, the constraints, what to run, and what to show me, e.g. *"Stage 1: semantic_search and keyword_search... named parameters everywhere; no string formatting of user input. Run 2 queries and print the top 5. Don't commit."*
- **Scope boundaries.** Most prompts named what *not* to do yet ("no word clean-up or speaker splitting yet, I'll add those as a separate step"), which kept each change small enough to review.

I kept control of the repository.

- **Commits were mine.** Every step ended with "don't commit": I checked the output and the changes, then committed in small steps myself, so the history shows the system being built up piece by piece.
- **Guard rails in the prompt:** use my earlier prototype as a reference only and never copy from it (nor its SSH keys); secrets come only from the environment. Destructive steps, such as recreating the database, happened only after the agent confirmed there was no data to lose and I gave the go-ahead.

I asked for evidence, not claims.

- **Show real output.** Each step had to run on real data and show the result: first transcript segments, chunk summaries, database row counts, search results for test queries.
- **Test on a copy first.** When I asked it to verify my Postgres schema, it applied the schema twice to a throwaway database and ran keyword and vector queries against real chunks before I applied it to mine.
- **Clean-room check before submission.** I had it set the project up from a clean copy of the repo (fresh virtual environment and database). This caught an incorrect Python version in the README, unpinned dependencies, and a crash in the evaluation CLI.

I used it as a reviewer, and the decisions stayed mine.

- **Reviews on request:** my SQL schema (it found redundant indexes and a missing chunk id), the codebase for submission readiness (duplicated paths, dead code, noisy logs), and my prototype's success criteria against the new results.

- **Examples of my calls:** renaming the container and database; recreating the database with the schema as an init script; committing the audio dataset; which review fixes to apply now and which to defer until the evaluation existed; and which retrieval change to keep.

Measure before changing anything.

- **Baseline and comparisons.** For the retrieval improvements I had it freeze a baseline, build a comparison report, and score several variants, reporting all of them rather than only the winner (section 7.3).
- **No gaming the metric.** When recall@5 was below target, the agent pointed out that adding easy queries would raise the number without improving search. We improved retrieval instead, and recorded that the final setting was chosen on the same query set.
- **Evaluation data.** The agent drafted the 25 queries from the transcripts and I reviewed them. A validator flags "semantic" queries that reuse the speaker's own words; 4 were reworded.

Where the agent was wrong, and how it was caught. Verification caught the agent's own mistakes:

- **Evaluation.** In the prototype, a ± 2 s match tolerance counted the other speaker's turn as correct, and de-duplicating neighbouring results hid the right answer (issues 7 and 8).
- **During the rebuild:**
 - a refactor line that would have attached words to the wrong speaker, caught by comparing old and new code on 5,000 random segments;
 - a database test that looked right but never committed;
 - a timestamp formatter that printed 59.96 s as "00:60.0".

Reusable context for future sessions. At the end I captured the working setup so the next session doesn't start from zero:

- **CLAUDE.md:** project layout, commands and conventions, e.g. "schema changes need a re-index", "never commit .env", "don't edit labeled queries to move a metric".
- **A project skill, .claude/skills/retrieval-experiment:** encodes the evaluate → compare → log loop used in section 7.3.